

Working with Subqueries

In this chapter, you'll learn what subqueries are and how to use them.

Understanding Subqueries

`SELECT` statements are SQL queries. All the `SELECT` statements you have seen thus far are simple queries: single statements that retrieve data from individual database tables.

New Term

Query Any SQL statement. However, the term is usually used to refer to `SELECT` statements.

SQL also enables you to create *subqueries*: queries that are embedded in other queries. Why would you want to do this? The best way to understand this concept is to look at a couple of examples.

Filtering by Subquery

The database tables used in all the chapters in this book are relational tables. (See Appendix B, “The Example Tables,” for a description of each of the tables and their relationships.) Order data is stored in two tables. The `orders` table stores a single row for each order and contains the order number, customer ID, and order date. The individual order items are stored in the related `orderitems` table. The `orders` table does not store any customer information except for the customer ID. The `customers` table stores the other customer information.

Suppose you want a list of all the customers who have ordered item TNT2. What do you have to do to retrieve this information? Here are the steps:

1. Retrieve the order numbers of all orders containing item TNT2.
2. Retrieve the customer ID of every customer who has an order in the list of order numbers returned in step 1.
3. Retrieve the customer information for each customer ID returned in step 2.

Each of these steps can be executed as a separate query. You can use the results returned by one `SELECT` statement to populate the `WHERE` clause of the next `SELECT` statement.

You can also use subqueries to combine all three queries into a single statement.

The `SELECT` statement shown here should be self-explanatory by now. It retrieves the `order_num` column for every order item with `prod_id` of `TNT2`. The output lists the two orders containing this item:

► Input

```
SELECT order_num
FROM orderitems
WHERE prod_id = 'TNT2';
```

► Output

```
+-----+
| order_num |
+-----+
|      20005 |
|      20007 |
+-----+
```

The next step is to retrieve the customer IDs associated with orders 20005 and 20007. Using the `IN` clause described in Chapter 7, “Advanced Data Filtering,” you can create a `SELECT` statement as follows:

► Input

```
SELECT cust_id
FROM orders
WHERE order_num IN (20005,20007);
```

► Output

```
+-----+
| cust_id |
+-----+
|    10001 |
|    10004 |
+-----+
```

Now, combine the two queries by turning the first (the one that returned the order numbers) into a subquery. Look at the following `SELECT` statement:

► Input

```
SELECT cust_id
FROM orders
WHERE order_num IN (SELECT order_num
                     FROM orderitems
                     WHERE prod_id = 'TNT2');
```

► Output

```
+-----+  
| cust_id |  
+-----+  
| 10001 |  
| 10004 |  
+-----+
```

► Analysis

Subqueries are always processed starting with the innermost `SELECT` statement and working outward. When this `SELECT` statement is processed, MySQL actually performs two operations. First, it runs the subquery:

```
| SELECT order_num  
| FROM orderitems  
| WHERE prod_id='TNT2'
```

This query returns the two order numbers 20005 and 20007. Those two values are then passed to the `WHERE` clause of the outer query in the comma-delimited format required by the `IN` operator. The outer query now becomes the following:

```
| SELECT cust_id  
| FROM orders  
| WHERE order_num IN (20005,20007)
```

As you can see, the output is correct and exactly the same as the output returned by the previous hard-coded `WHERE` clause.

Tip

Formatting Your SQL `SELECT` statements that contain subqueries can be difficult to read and debug, especially as they grow in complexity. Breaking up the queries over multiple lines and indenting the lines appropriately, as shown here, can greatly simplify working with subqueries.

You now have the IDs of all the customers who ordered item TNT2. The next step is to retrieve the customer information for each of those customer IDs. The SQL statement to retrieve the two columns is as follows:

► Input

```
| SELECT cust_name, cust_contact  
| FROM customers  
| WHERE cust_id IN (10001,10004);
```

Instead of hard-coding those customer IDs, you can turn this `WHERE` clause into yet another subquery:

► **Input**

```
SELECT cust_name, cust_contact
FROM customers
WHERE cust_id IN (SELECT cust_id
                  FROM orders
                  WHERE order_num IN (SELECT order_num
                                      FROM orderitems
                                      WHERE prod_id = 'TNT2'));
```

► **Output**

```
+-----+-----+
| cust_name | cust_contact |
+-----+-----+
| Coyote Inc. | Y Lee      |
| Yosemite Place | Y Sam    |
+-----+-----+
```

► **Analysis**

To execute this `SELECT` statement, MySQL had to actually perform three `SELECT` statements. The innermost subquery returned a list of order numbers that were then used as the `WHERE` clause for the subquery above it. That subquery returned a list of customer IDs that were used as the `WHERE` clause for the top-level query. The top-level query actually returned the desired data.

As you can see, using subqueries in a `WHERE` clause enables you to write extremely powerful and flexible SQL statements. There is no limit on the number of subqueries that can be nested, although in practice you will find that performance tells you when you are nesting too deeply.

Caution

Columns Must Match When using a subquery in a `WHERE` clause (as shown here), make sure that the `SELECT` statements have the same number of columns as in the `WHERE` clause. Usually, a single column will be returned by the subquery and matched against a single column, but multiple columns may be used, if needed.

Although usually used in conjunction with the `IN` operator, subqueries can also be used to test for equality (using `=`), non-equality (using `<>`), and so on.

Caution

Subqueries and Performance The code shown here works, and it achieves the desired result. However, using subqueries is not always the most efficient way to perform this type of data retrieval—though sometimes it is. More on this is in Chapter 15, “Joining Tables,” where we will revisit this example.

Using Subqueries As Calculated Fields

Another way to use subqueries is in creating calculated fields. Suppose you want to display the total number of orders placed by every customer in your `customers` table. Orders are stored in the `orders` table along with the appropriate customer IDs.

To perform this operation, follow these steps:

1. Retrieve the list of customers from the `customers` table.
2. For each customer retrieved, count the number of associated orders in the `orders` table.

As you learned in the previous two chapters, you can use `SELECT Count(*)` to count rows in a table, and by providing a `WHERE` clause to filter a specific customer ID, you can count just that customer's orders. For example, the following code counts the number of orders placed by customer 10001:

► Input

```
SELECT Count(*) AS orders
FROM orders
WHERE cust_id = 10001;
```

To perform this `Count(*)` calculation for each customer, you can use `Count(*)` as a subquery. Look at the following code:

► Input

```
SELECT cust_name,
       cust_state,
       (SELECT COUNT(*)
        FROM orders
        WHERE orders.cust_id = customers.cust_id) AS orders
FROM customers
ORDER BY cust_name;
```

► Output

cust_name	cust_state	orders
Coyote Inc.	MI	2
E Fudd	IL	1
Mouse House	OH	0
Wascals	IN	1
Yosemite Place	AZ	1

► Analysis

This `SELECT` statement returns three columns for every customer in the `customers` table: `cust_name`, `cust_state`, and `orders`. `orders` (which is a calculated field that is set by a subquery provided in parentheses). This subquery is executed once for every

customer retrieved. In this example, the subquery is executed five times because five customers are retrieved.

The `WHERE` clause in the subquery is a little different from the `WHERE` clauses used previously because it uses fully qualified column names (first mentioned in Chapter 4, “Retrieving Data”). The following clause tells SQL to compare the `cust_id` in the `orders` table to the one currently being retrieved from the `customers` table:

```
| WHERE orders.cust_id = customers.cust_id
```

This type of subquery is called a *correlated subquery*.

New Term

Correlated Subquery A subquery that refers to the outer query.

This syntax—the table name and the column name separated by a period—must be used whenever there is possible ambiguity about column names. Why? Well, let’s look at what happens if fully qualified column names are not used:

► Input

```
SELECT cust_name,
       cust_state,
       (SELECT Count(*)
        FROM orders
        WHERE cust_id = cust_id) AS orders
FROM customers
ORDER BY cust_name;
```

► Output

cust_name	cust_state	orders
Coyote Inc.	MI	5
E Fudd	IL	5
Mouse House	OH	5
Wascals	IN	5
Yosemite Place	AZ	5

► Analysis

Obviously, the returned results are incorrect; to see this, compare them to the previous results. Why did this happen? There are two `cust_id` columns, one in `customers` and one in `orders`, and those two columns need to be compared to correctly match orders with the appropriate customers. If you don’t fully qualify the column names, MySQL assumes that you are comparing the `cust_id` in the `orders` table to itself. And this

statement always returns the total number of orders in the `orders` table (because MySQL checks to see that every order's `cust_id` matches itself, which it always does, of course):

```
| SELECT Count(*) FROM orders WHERE cust_id = cust_id;
```

Although subqueries are extremely useful in constructing this type of `SELECT` statement, care must be taken to properly qualify ambiguous column names.

Note

Always More Than One Solution As explained earlier in this chapter, although the sample code shown here works, this is often not the most efficient way to perform this type of data retrieval. We will revisit this example in Chapter 15.

Tip

Build Queries with Subqueries Incrementally Testing and debugging queries with subqueries can be tricky, particularly as these statements grow in complexity. The safest way to build (and test) queries with subqueries is to do so incrementally, in much the same way that MySQL processes them. Build and test the innermost query first. Then build and test the outer query with hard-coded data, and only after you have verified that it is working, embed the subquery. Then test it again. Keep repeating these steps for each additional query. It will take just a little longer to construct your queries if you do it this way, but it will save you lots of time later when you need to try to figure out why queries are not working. It will also significantly increase the likelihood that the queries will work the first time.

Summary

In this chapter, you learned what subqueries are and how to use them. The most common uses for subqueries are in `WHERE` clauses, in `IN` operators, and for populating calculated columns. You saw examples of both of these types of operations.

Challenges

1. Using a subquery, return a list of customers who bought items priced 10 or more. You'll want to use the `OrderItems` table to find the matching order numbers (`order_num`) and then the `Orders` table to retrieve the customer ID (`cust_id`) for each matched order.
2. You need to know the dates when product BR01 was ordered. Write a SQL statement that uses a subquery to determine which orders (in `OrderItems`) purchased items with `prod_id` of BR01 and then returns the customer ID (`cust_id`)

and order date (`order_date`) for each of them from the `Orders` table. Sort the results by order date.

3. Now let's make it a bit more challenging. Update the previous challenge to return the customer email (`cust_email` in the `Customers` table) for any customer who purchased an item with `prod_id` of `BR01`. Here's a hint: This involves the `SELECT` statement, the innermost query returning `order_num` from `OrderItems`, and the middle query returning `cust_id` from `Customers`.
4. You need a list of customer IDs with the total amount each customer has ordered. Write a SQL statement that returns the customer ID (`cust_id` in the `Orders` table) and `total_ordered` and uses a subquery to return the total of orders for each customer. Sort the results by the amount spent from greatest to the least. Here's a hint: You've used `SUM()` to calculate order totals previously.
5. Write a SQL statement that retrieves all product names (`prod_name`) from the `Products` table, along with a calculated column named `quant_sold` that contains the total number of this item sold (retrieved using a subquery and `SUM(quantity)` on the `OrderItems` table).